

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (UMBRELLO UML MODELLER)
Ciclo MDD: modelo UML, generación de código C#, verificación y roundtrip

DOCUMENTO INDIVIDUAL:

INVESTIGACIÓN INDIVIDUAL - ANGELO ZAMBRANO

ASIGNATURA:

INGENIERÍA DE REQUISITOS (ISR-401)

DOCENTE:

ING. GLEISTON GUERRERO ULLOA, PhD.

ESTUDIANTE:

ZAMBRANO MOYA ANGELO PAUL

RESPONSABILIDAD PRINCIPAL:

Introducción general, configuración del generador, generación C#, verificación, roundtrip y coordinación técnica del repositorio.

EQUIPO:

A

CURSO/PARALELO:

CUARTO NIVEL PARALELO “B”

SISTEMA PFC:

SISTEMA INTELIGENTE DE CONTROL Y SEGUIMIENTO DE TERAPIA FÍSICA

HERRAMIENTA MDD Y LENGUAJE OBJETIVO:

UMBRELLO UML MODELLER – C#

REPOSITORIO GITHUB:

<https://github.com/AngeloP2114/SISTEMA-DE-TERAPIA-EQUIPOA-UMBRELLO-UML/tree/main>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1. Resumen	1
2. Introducción	1
3. Fundamento de la generación de código	1
3.1. Transformación modelo-a-texto.....	1
3.2. Forward engineering y roundtrip controlado	1
4. Preparación del entorno	2
4.1. Modelo de entrada.....	2
4.2. Dependencias de verificación.....	2
5. Selección de clases en el asistente	2
6. Configuración del generador C#	3
6.1. La diferencia entre las dos rutas	4
7. Ejecución y evidencia de generación	5
8. Integración y verificación en Visual Studio	5
8.1. Código generado y código manual.....	6
8.2. Correspondencia modelo-código.....	6
8.3. Unidad mínima de ejecución	7
9. Incidencia técnica: clase Ejercicio repetida	7
10. Roundtrip controlado	7
11. Evaluación de calidad	8
12. Reproducibilidad, ética y repositorio	8
13. Aporte personal de Angelo	8
14. Conclusiones	8
Apéndice A: Guion personal para la demostración	9
Apéndice B: Preguntas técnicas que Angelo debe dominar	9

1. Resumen

Esta investigación documenta la contribución de Angelo Zambrano al ciclo de desarrollo dirigido por modelos aplicado al Sistema Inteligente de Control y Seguimiento de Terapia Física. El trabajo se concentra en la transición desde el modelo UML construido en Umbrello UML Modeller hacia código fuente C#, la configuración del asistente de generación, la verificación de los archivos producidos, la diferenciación entre código generado y código manual, y el cierre del ciclo mediante una regeneración controlada. La evidencia práctica incluye la selección explícita de siete clases del dominio, la configuración del lenguaje C#, la definición de la carpeta de salida, el estado exitoso de generación para cada clase y la integración de los archivos en Visual Studio. También se analiza una incidencia real ocurrida durante las primeras pruebas: la repetición de la clase `Ejercicio` en todos los archivos debido al uso incorrecto de la opción de encabezados. La investigación concluye que Umbrello aporta una generación estructural útil y trazable, pero que no reemplaza la implementación de reglas de negocio, las pruebas ni la revisión humana. El aporte de Angelo abarca además la introducción del trabajo, la preparación de la demostración, la política de regeneración y la organización técnica necesaria para reproducir el proceso desde GitHub.

2. Introducción

La Ingeniería Dirigida por Modelos plantea que los modelos de software deben desempeñar un papel activo durante el desarrollo y no limitarse a ser diagramas elaborados para la documentación. En este enfoque, la estructura formal representada mediante UML puede ser procesada por herramientas capaces de generar artefactos textuales, entre ellos código fuente. Esta transformación reduce la repetición de tareas mecánicas, mejora la uniformidad de la estructura y permite comprobar de manera directa la correspondencia entre el diseño y la implementación [1-3].

La actividad desarrollada por el Equipo A parte de un subsistema del PFC de terapias físicas. El modelo contiene las clases `Usuario`, `Paciente`, `Fisioterapeuta`, `PlanTerapia`, `Ejercicio`, `SesionTerapia` y `EvaluacionProgreso`. A partir de este modelo se ejecuta una transformación modelo-a-texto hacia C#, se incorporan los archivos al entorno de Visual Studio y se documentan sus límites. La guía de evaluación exige que la generación sea verificable, que el equipo distinga el código producido por la herramienta del código añadido manualmente, que exista correspondencia modelo-código y que se realice una regeneración controlada [4].

Dentro de la distribución grupal, Angelo asumió el bloque de mayor responsabilidad operativa: introducir el problema, explicar el propósito de la generación, preparar el entorno, configurar el asistente, ejecutar la generación, revisar los archivos, coordinar la verificación en C# y explicar el manejo de la regeneración. Esta función es crítica porque conecta las actividades previas de fundamentación y modelado con la evidencia práctica que demuestra que Umbrello posee capacidad real de producir código.

3. Fundamento de la generación de código

3.1. Transformación modelo-a-texto

El proceso realizado corresponde a una transformación M2T, es decir, una transformación en la que el artefacto de entrada es un modelo y el resultado es texto estructurado. En el caso analizado, las clases UML son interpretadas por el generador interno de Umbrello y se convierten en declaraciones C#. Los nombres de las clases producen archivos, los atributos producen campos, las operaciones producen firmas de métodos y la generalización produce herencia en el lenguaje objetivo. El estándar MOFM2T establece una base conceptual para este tipo de transformaciones, aunque Umbrello emplea su propio motor integrado [5, 6].

La transformación no debe confundirse con la generación de una aplicación terminada. La salida contiene principalmente estructuras del dominio. Una operación UML como `calcularProgreso(...): double` puede convertirse en una firma de C#, pero la fórmula que determina el progreso requiere implementación. Esta diferencia es esencial para evaluar honestamente la herramienta y evita presentar esqueletos estructurales como un sistema completamente funcional.

3.2. Forward engineering y roundtrip controlado

La generación ejecutada constituye *forward engineering*: el modelo es la fuente desde la que se produce código. Umbrello también dispone de funciones de importación, pero la sincronización bidireccional no debe equipararse a la de plataformas comerciales avanzadas. La documentación oficial indica que, si un archivo ya existe, el generador puede sobrescribirlo, omitirlo o producir otro nombre; no realiza una fusión semántica general de cambios manuales [6, 7].

Por esta razón, el cierre del ciclo se diseñó como un roundtrip controlado y unidireccional: se modifica un elemento UML, se regenera la clase afectada y se verifica que el cambio aparezca en el archivo correspondiente. Esta forma de trabajo conserva la trazabilidad y evita afirmar una sincronización que la herramienta no garantiza.

4. Preparación del entorno

4.1. Modelo de entrada

Antes de utilizar el generador se revisó que las siete clases fueran reconocidas como clases del dominio y que los tipos auxiliares, como `DateTime` y `void`, no se seleccionaran como unidades de generación. También se verificó la visibilidad: los atributos permanecieron privados y las operaciones públicas. Esta decisión representa el principio de encapsulamiento y facilita que el código generado mantenga una estructura orientada a objetos coherente.

La revisión previa incluyó los siguientes puntos:

- nombres de clases sin espacios ni caracteres que dificulten la generación;
- atributos con tipo explícito;
- operaciones con tipo de retorno;
- parámetros definidos mediante nombre y tipo;
- herencia de `Paciente` y `Fisioterapeuta` respecto de `Usuario`;
- relaciones todo–parte representadas mediante agregación o composición;
- eliminación o renombrado de elementos temporales como `new_attribute`.

4.2. Dependencias de verificación

Para revisar el resultado se utilizó un proyecto de consola C# en Visual Studio. El generador produce archivos `.cs`, mientras que el entorno .NET proporciona el compilador y el punto de entrada necesario para ejecutar una unidad mínima. El comando `dotnet run` o la opción de ejecución de Visual Studio compilan y ejecutan el proyecto, pero requieren que exista un archivo de inicio, normalmente `Program.cs` [8].

Es importante aclarar que `Program.cs` no fue generado por Umbrello. Se añadió manualmente para validar la integración de las clases. Esta separación se documentó para cumplir el requisito ético de no presentar código manual como salida automática.

5. Selección de clases en el asistente

El asistente *Code Generation Wizard* permite controlar qué elementos del modelo producirán archivos. La Figura 1 muestra la selección real utilizada durante la generación. En la columna derecha aparecen las siete clases del dominio; en la izquierda permanecen `DateTime` y `void`, que son tipos auxiliares y no deben convertirse en archivos independientes.

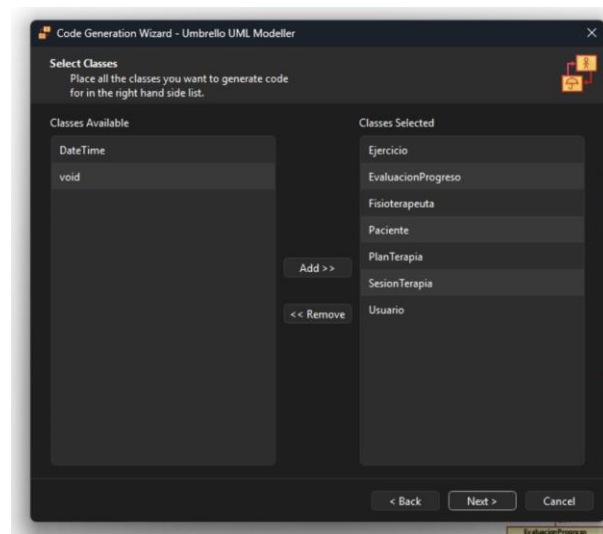


Figura 1: Selección de las siete clases del dominio en el asistente de Umbrello.

La selección explícita aporta dos evidencias. Primero, confirma que el código procede del modelo real y no de archivos preparados manualmente. Segundo, permite reproducir el proceso: cualquier integrante puede abrir el modelo, seleccionar las mismas clases y obtener el mismo conjunto de archivos.

Tabla 1: Clases seleccionadas y archivo esperado

Clase UML	Archivo esperado	Responsabilidad principal
Usuario	Usuario.cs	Datos comunes y acceso al sistema.
Paciente	Paciente.cs	Información clínica y consulta de progreso.
Fisioterapeuta	Fisioterapeuta.cs	Evaluación y creación de planes.
PlanTerapia	PlanTerapia.cs	Organización del tratamiento.
Ejercicio	Ejercicio.cs	Actividades, repeticiones y duración.
SesionTerapia	SesionTerapia.cs	Registro y seguimiento de cada sesión.
EvaluacionProgreso	EvaluacionProgreso.cs	Medición del progreso del paciente.

6. Configuración del generador C#

La Figura 2 registra las opciones reales del asistente. El lenguaje seleccionado fue C#. La ruta *Write all generated files to folder* corresponde al destino donde Umbrello crea los archivos. La opción *Include heading files from folder* aparece desactivada, aunque el campo conserva una ruta previa; al estar la casilla sin marcar, esa ruta no debe intervenir en la generación.

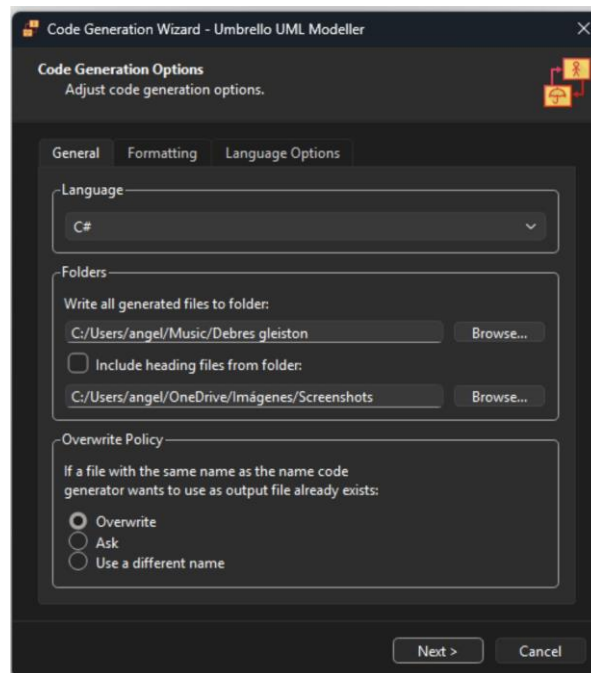


Figura 2: Configuración de C#, carpeta de salida y política de sobrescritura.

En la corrida capturada se utilizó la política *Overwrite*. Esta configuración reemplaza archivos del mismo nombre y es útil cuando se desea regenerar completamente una salida. No obstante, para una demostración académica o un proyecto con modificaciones manuales, la política recomendada es *Ask*, porque obliga a confirmar cada sustitución y disminuye el riesgo de pérdida accidental. La diferencia se explica en la Tabla 2.

Tabla 2: Políticas disponibles ante archivos existentes

Política	Comportamiento	Uso recomendado
Overwrite	Reemplaza el archivo existente.	Regeneración limpia de código sin ediciones manuales.
Ask	Solicita confirmación antes de reemplazar.	Demostraciones y proyectos con revisión humana.
Use a different name	Conserva el archivo y crea otra versión.	Comparaciones antes/después o respaldo.

6.1. La diferencia entre las dos rutas

La primera ruta es obligatoria: identifica la carpeta de salida. La segunda ruta no es una carpeta adicional de generación. Su propósito es insertar encabezados comunes, por ejemplo una licencia o aviso institucional, al inicio de cada archivo. Activarla con una carpeta que contenga código provoca que ese contenido se copie repetidamente. Esta distinción fue uno de los aprendizajes técnicos principales de la práctica.

7. Ejecución y evidencia de generación

La Figura 3 combina dos evidencias. En la parte superior, el asistente informa *Code Generated* para cada clase; en la parte inferior, el explorador de archivos muestra los siete archivos `.cs`. La coincidencia entre la lista del asistente y la carpeta de salida permite afirmar que Umbrello realizó la generación de forma verificable.

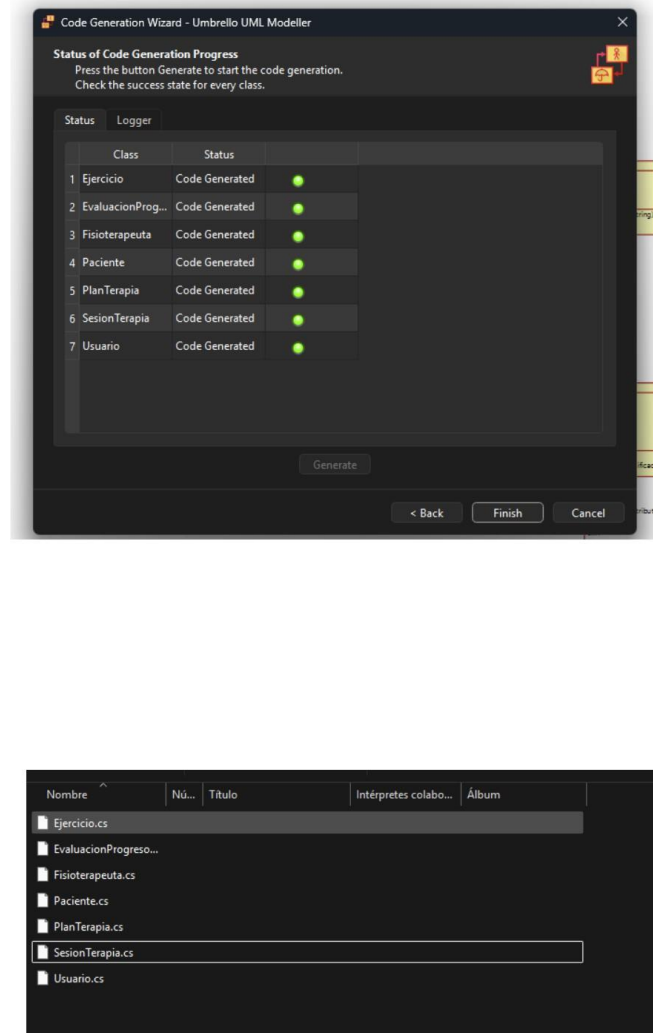


Figura 3: Estado exitoso del generador y archivos C# producidos.

El proceso puede describirse como la cadena:

modelo UML → selección de clases → configuración C# → archivos `.cs`

La evidencia resulta más sólida que una muestra aislada de código, porque registra el momento de generación y los archivos de salida. Además, el repositorio debe conservar el modelo nativo, las capturas, la configuración y el código para que otra persona pueda repetir la corrida, tal como exige la guía [4].

8. Integración y verificación en Visual Studio

La Figura 4 muestra los archivos incorporados al proyecto de consola. Se observan las siete clases producidas por Umbrello y el archivo `Program.cs`. Esta estructura permite diferenciar claramente el dominio generado de la

unidad manual de prueba.

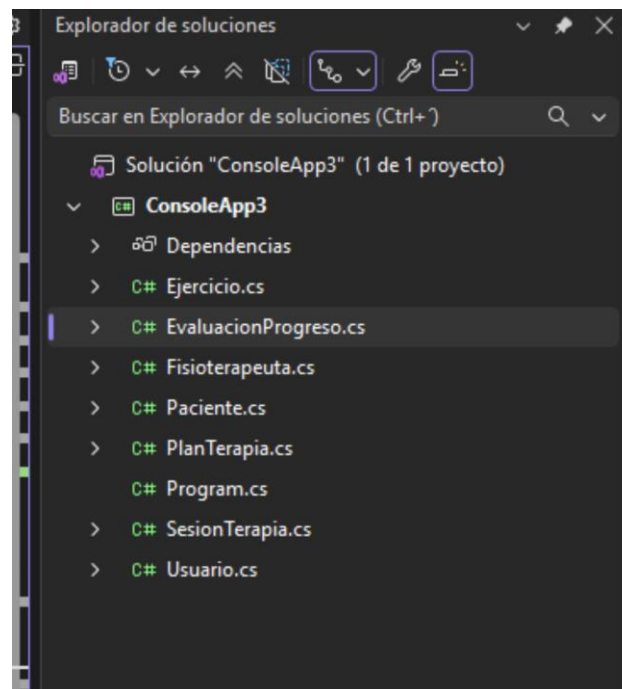


Figura 4: Archivos generados incorporados a un proyecto de verificación en Visual Studio.

8.1. Código generado y código manual

La clasificación adoptada se presenta a continuación:

- **Generado:** archivos de las siete clases, campos, firmas de operaciones y herencia derivada del modelo.
- **Manual de verificación:** `Program.cs`, creación de objetos, mensajes en consola y lógica mínima necesaria para ejecutar una prueba.
- **Ajustes mínimos:** retornos de prueba para métodos no `void`, cuando el generador produce una firma sin cuerpo funcional.

La distinción evita una interpretación incorrecta del resultado. La herramienta genera una línea base estructural, no la totalidad de la aplicación.

8.2. Correspondencia modelo–código

Tabla 3: Trazabilidad entre elementos UML y C#

Elemento UML	Artefacto C#	Comprobación
Clase Paciente	<code>Paciente.cs</code>	Archivo independiente con la declaración de clase.
Generalización a Usuario	<code>Paciente : Usuario</code>	La herencia refleja el triángulo de generalización.
<code>edad:int</code>	Campo entero privado	Conserva nombre, tipo y visibilidad.
<code>actualizarDiagnostico(...):Método público</code>	<code>PlanTerapia.cs</code>	Conserva la firma y el parámetro.
Clase PlanTerapia	<code>PlanTerapia.cs</code>	Contiene fechas, objetivo, estado y operaciones.
Parámetro	Parámetro de tipo <code>Ejercicio</code>	Recibe un objeto completo del dominio.
<code>ejercicio:Ejercicio</code>		

8.3. Unidad mínima de ejecución

Una prueba manual puede instanciar clases y utilizar una operación de progreso. El siguiente ejemplo no debe presentarse como generado; funciona únicamente como evidencia de integración.

Código 1: Unidad manual mínima para probar las clases generadas

```

1 // [MANUAL] Punto de entrada para verificación.
2 Paciente paciente = new Paciente();
3 Fisioterapeuta fisioterapeuta = new Fisioterapeuta();
4 PlanTerapia plan = new PlanTerapia();
5 Ejercicio ejercicio = new Ejercicio();
6 SesionTerapia sesion = new SesionTerapia();
7
8 fisioterapeuta.evaluarPaciente(paciente);
9 plan.agregarEjercicio(ejercicio);
10 sesion.finalizarSesion();
11
12 Console.WriteLine("Clases generadas integradas correctamente.");

```

9. Incidencia técnica: clase Ejercicio repetida

Durante una prueba inicial, la clase `Ejercicio` apareció al comienzo de varios archivos y, posteriormente, aparecía la clase que realmente correspondía al archivo. El problema no era el parámetro `ejercicio:Ejercicio`; esa firma es válida porque el primer término es el nombre de la variable y el segundo es su tipo. La causa estaba en la opción de encabezados.

Se había señalado una carpeta con código como origen de encabezados. Umbrello interpretó el contenido como texto común y lo insertó en todas las salidas. La corrección se realizó mediante los siguientes pasos:

1. crear una carpeta de salida nueva y vacía;
2. desmarcar *Include heading files from folder*;
3. conservar únicamente la ruta *Write all generated files to folder*;
4. regenerar las siete clases;
5. comprobar que `public class Ejercicio` apareciera solo en `Ejercicio.cs`.

Este incidente demuestra por qué la configuración forma parte de la evaluación. Una herramienta puede generar código incorrecto no por un error en el modelo, sino por una opción del generador mal interpretada.

10. Roundtrip controlado

El cambio propuesto para cerrar el ciclo consiste en agregar un atributo clínico a `EvaluacionProgreso`, por ejemplo:

```
- fatiga: int
```

El procedimiento es:

1. capturar la clase antes del cambio;
2. agregar el atributo privado en Umbrello;
3. seleccionar solo `EvaluacionProgreso` en el asistente;
4. regenerar en la misma carpeta con política controlada;
5. abrir `EvaluacionProgreso.cs`;
6. verificar la aparición del nuevo campo;
7. compilar nuevamente el proyecto.

El cambio es más significativo que una modificación ortográfica porque amplía la información clínica del modelo. La evidencia antes/después conecta directamente la evolución del modelo con el archivo generado.

11. Evaluación de calidad

Tabla 4: Evaluación del resultado generado

Criterio	Resultado	Análisis
Correspondencia estructural	Alta	Se producen clases, campos, métodos y herencia acordes al modelo.
Compleitud funcional	Baja	Las reglas de negocio y persistencia no son generadas.
Legibilidad	Media-alta	El código se organiza por clase, aunque depende de las opciones de formato.
Compilabilidad	Condicionada	Los métodos con retorno pueden necesitar implementación mínima.
Trazabilidad	Alta	Es posible relacionar clases y miembros con su origen UML.
Seguridad ante regeneración	Media	Debe controlarse la sobrescritura y separar el código manual.

La calidad del código generado no puede medirse solo por la ausencia de errores sintácticos. También debe evaluarse la fidelidad al modelo, la facilidad de mantenimiento, la claridad de los límites y la reproducibilidad. La investigación sobre MDE identifica precisamente la evolución, la interoperabilidad y la adopción como desafíos persistentes [9, 10].

12. Reproducibilidad, ética y repositorio

El repositorio público constituye la evidencia técnica central. Debe contener el modelo nativo, las capturas, el código generado, la unidad manual de verificación, el informe y un README con instrucciones. La contribución individual de Angelo se demuestra mediante commits propios sobre configuración, generación, verificación, documentación y preparación de la exposición.

Los criterios éticos aplicados son:

- no declarar como generado el archivo `Program.cs`;
- identificar ajustes manuales;
- no ocultar errores de configuración;
- revisar el código antes de su uso;
- conservar la trazabilidad y el historial de cambios;
- no atribuir a Umbrello lógica que fue implementada por el equipo.

13. Aporte personal de Angelo

El aporte individual comprende:

1. redacción de la introducción y presentación del propósito del ciclo UML–código;
2. selección y verificación de C# como lenguaje objetivo;
3. ejecución del *Code Generation Wizard*;
4. análisis de las opciones de carpeta y encabezados;
5. diagnóstico y corrección del error de Ejercicio repetido;
6. incorporación de archivos a Visual Studio;
7. diferenciación entre código generado y manual;
8. diseño de la unidad mínima de verificación;
9. planificación del roundtrip controlado;
10. organización de la evidencia técnica y coordinación del repositorio.

Estas tareas concentran la parte operativa y demostrativa exigida por la rúbrica, por lo que justifican que Angelo participe en la introducción, el segmento principal de generación y el cierre técnico de la exposición.

14. Conclusiones

Umbrello demostró capacidad real para transformar un modelo UML del PFC en un conjunto de archivos C#. La generación fue verificable mediante la selección de clases, la configuración del asistente, el estado exitoso de cada elemento y la presencia de los archivos en la carpeta de salida. La integración en Visual Studio confirmó que el resultado puede utilizarse como base de un proyecto.

El mayor aprendizaje fue comprender que la generación no elimina la responsabilidad del desarrollador. Las firmas y estructuras provienen del modelo, mientras que la lógica, las pruebas y el punto de entrada deben completarse. La incidencia del encabezado mostró que una configuración incorrecta puede afectar todos los archivos, y que el conocimiento de las opciones es tan importante como el diagrama.

Finalmente, el valor de MDD para el sistema de terapias físicas se encuentra en la consistencia estructural, la trazabilidad y la reducción de trabajo repetitivo. El proceso no sustituye la ingeniería de software; proporciona una base controlada sobre la que el equipo puede implementar las reglas clínicas y tecnológicas del PFC.

Apéndice A

Guion personal para la demostración

La participación de Angelo durante la exposición puede organizarse en un hilo continuo que conecte teoría, configuración y evidencia. El siguiente guion permite demostrar dominio sin limitarse a presionar el botón de generación.

Tabla 5: Secuencia recomendada para la intervención de Angelo

Tiempo	Acción visible	Explicación
0:00–0:40	Presentar el objetivo.	“El modelo UML será transformado en archivos C# y luego verificado en Visual Studio.”
0:40–1:20	Mostrar el modelo.	Identificar clases, tipos, herencia y operaciones que se esperan en el código.
1:20–2:10	Abrir el asistente.	Explicar por qué se usa <i>Code Generation Wizard</i> y no una exportación de imagen.
2:10–2:50	Seleccionar clases.	Confirmar que <code>DateTime</code> y <code>void</code> no se generan como clases.
2:50–3:40	Configurar C#.	Diferenciar la carpeta de salida y la opción de encabezados.
3:40–4:20	Generar.	Interpretar el estado <i>Code Generated</i> de cada clase.
4:20–5:10	Abrir Visual Studio.	Comparar <code>Paciente</code> y <code>PlanTerapia</code> con el modelo.
5:10–6:00	Evaluar calidad.	Diferenciar estructura generada, lógica manual y limitaciones.
6:00–7:00	Roundtrip.	Agregar <code>fatiga:int</code> , regenerar y mostrar el cambio.

Apéndice B

Preguntas técnicas que Angelo debe dominar

1. **¿Por qué la salida no se ejecuta directamente?** Porque Umbrello genera clases del dominio, pero no crea necesariamente un punto de entrada ni la lógica completa.
2. **¿Qué diferencia existe entre la carpeta de salida y la carpeta de encabezados?** La primera recibe los archivos generados; la segunda inserta contenido común al inicio de cada archivo y normalmente debe permanecer desactivada.
3. **¿Por qué `Ejercicio:Ejercicio` es correcto?** Porque el parámetro se llama `Ejercicio` y su tipo es la clase `Ejercicio`.
4. **¿Qué evidencia prueba que el código fue generado?** La selección de clases, el estado exitoso del asistente, la carpeta de salida y la correspondencia con el modelo.
5. **¿Qué ocurre al regenerar?** Dependiendo de la política, el archivo se sobrescribe, se conserva o se genera con otro nombre; no existe una fusión automática general.
6. **¿Cuál fue el error más importante?** La inserción de `Ejercicio` como encabezado común por una configuración incorrecta.
7. **¿Qué parte es manual?** `Program.cs`, las reglas de negocio y los retornos mínimos necesarios para compilar.

Referencias

- [1] D. C. Schmidt, “Guest Editor’s Introduction: Model-Driven Engineering,” *Computer*, vol. 39, n.º 2, págs. 25-31, 2006. DOI: 10.1109/MC.2006.58.
- [2] B. Selic, “The Pragmatics of Model-Driven Development,” *IEEE Software*, vol. 20, n.º 5, págs. 19-25, 2003. DOI: 10.1109/MS.2003.1231146.
- [3] M. Brambilla, J. Cabot y M. Wimmer, *Model-Driven Software Engineering in Practice*, 2.^a ed. Morgan & Claypool Publishers, 2017. DOI: 10.2200/S00751ED2V01Y201701SWE001.
- [4] G. G. Ulloa, *Guía y Rúbrica de Evaluación: Exposición-Demostración MDD (UML a Código)*, Universidad Técnica Estatal de Quevedo, 2026.
- [5] Object Management Group, “MOF Model to Text Transformation Language, Version 1.0,” Object Management Group, inf. téc. formal/2008-01-16, 2008. dirección: <https://www.omg.org/spec/MOFM2T/1.0>.
- [6] KDE Documentation Team. “Umbrello UML Modeller Handbook: Code Import and Code Generation,” visitado 25 de jul. de 2026. dirección: https://docs.kde.org/trunk_kf6/en/umbrello/umbrello/code-import-generation.html.

- [7] KDE Documentation Team. “Umbrello UML Modeller Handbook: Code Generation Settings,” visitado 25 de jul. de 2026. dirección: https://docs.kde.org/trunk_kf6/en/umbrello/umbrello/code-generation-settings.html.
- [8] Microsoft. “dotnet run command,” visitado 25 de jul. de 2026. dirección: <https://learn.microsoft.com/dotnet/core/tools/dotnet-run>.
- [9] A. Bucchiarone, J. Cabot, R. F. Paige y A. Pierantonio, “Grand Challenges in Model-Driven Engineering: An Analysis of the State of the Research,” *Software and Systems Modeling*, vol. 19, n.º 1, págs. 5-13, 2020. DOI: 10.1007/s10270-019-00773-6.
- [10] J. Whittle, J. Hutchinson y M. Rouncefield, “The State of Practice in Model-Driven Engineering,” *IEEE Software*, vol. 31, n.º 3, págs. 79-85, 2014. DOI: 10.1109/MS.2013.65.